1. `main.py` - 專案編排器 (The Project Orchestrator)

用途： 此檔案是整個數據處理與機器學習流程的核心入口點與編排器。它定義了運算的執行順序、管理檔案路徑，並包含了提升開發效率的邏輯。

核心區段與邏輯：

- **路徑定義 (`SENSOR_DATA_PATH`, `POND_DAILY_LOGS_PATH` 等)：**
  - **目的：** 這些路徑定義了原始輸入資料（感測器讀數與每日日誌）的位置，以及處理後輸出檔案（如 `train_set.csv`, `test_set.csv`）的儲存位置。使用變數管理路徑可以提高程式碼的可維護性，並在檔案位置變更時輕鬆調整。
- **`main()` 函式：**
  - **目的：** 此函式封裝了整個工作流程。當 `main.py` 被直接執行時，系統會觸發此函式。
  - **條件式重複處理邏輯 (`if os.path.exists(...)`)：**
    - **目的：** 這是提升效率的關鍵功能。資料預處理與特徵工程通常涉及大量運算且耗時。此區塊會檢查最終的訓練集與測試集檔案是否已經存在。
    - **使用的函式：**
      - `os.path.exists()`: 檢查給定路徑的檔案或目錄是否存在。
      - `pd.read_csv()`: 若檔案已存在，使用 Pandas 快速載入為 DataFrame。
      - `pd.to_datetime()`: 將「日期」欄位轉回 datetime 物件，因為 CSV 儲存時會將其轉為字串，這對於時序運算至關重要。
    - **為何使用：** 避免冗餘運算。如果您已經處理過一次數據，除非原始數據或處理邏輯有變動，否則無需重新執行。這能顯著加快開發與測試週期。
- **呼叫 `build_features_and_split()`：**
  - **目的：** 若處理後的檔案不存在，此行將觸發整個特徵工程與數據切分流程。
  - **使用的函式：** 匯入自 `src.feature_builder.py` 的 `build_features_and_split()`。這是將原始數據轉換為最終訓練與測試資料集的主要函式。
- **階段 3：機器學習與 XGBoost (佔位符)：**
  - **目的：** 此部分清楚標示了專案的後續步驟（模型訓練、評估與可解釋性）。目前包含 TODO 註釋以引導未來開發。
  - **為何使用：** 明確定義 Pipeline 的各個階段，並根據技術文件規劃的結構，為加入機器學習組件提供清晰的框架。
- **`if __name__ == '__main__':` 區塊：**
  - **目的：** 這是標準的 Python 慣用語，確保 `main()` 函式僅在 `main.py` 被直接執行時運行。若該檔案被作為模組匯入其他腳本，則不會自動觸發。
  - **為何使用：** 使檔案兼具「可執行腳本」與「可匯入模組」的特性，促進程式碼重用且避免意外副作用。

---

2. `src/data_loader.py` - 數據基礎與初步清理

用途： 此檔案負責數據管線的基礎步驟：載入感測器與每日日誌原始資料、執行初步的時間對齊、轉換時間戳，並執行關鍵的數據完整性檢查與初步清理。這對應了預處理階段中的「數據對齊與預處理」。

核心函式：

- **`align_data(sensor_data_path, pond_daily_logs_path, min_records_per_day=10)`：**

- **目的：** 這是處理的第一步，接收原始 CSV 路徑，並回傳對齊且初步過濾後的 DataFrame。
- **使用的函式與原因：**
  - `pd.read_csv()`: 從 CSV 檔案載入數據，這是讀取表格數據的標準方式。
  - `pd.to_datetime()`: 將 `created_at`（感測器）與 `log_date`（日誌）欄位轉為 datetime 物件。這對於時序分析至關重要，能支援時區轉換、日期提取與排序。
  - **時區處理 (`tz_localize('UTC')` 與 `tz_convert('Asia/Taipei')`)：**
    - **目的：** IoT 設備通常以 UTC 或無時區資訊記錄。此步驟將其轉換為本地時區（台北），確保所有時間戳一致。
    - **為何使用：** 精確的時區處理可避免在聚合每日數據時出現「差一天」的錯誤，這對於將感測器讀數正確關聯到每日日誌事件至關重要。
  - **日期提取：** 從 `created_at` 中提取日期部分並轉回 datetime，以便將高頻率感測器數據與每日日誌進行合併（Merge）。
  - **狀態過濾 (`status == 'complete'`)：** 僅保留狀態為「完成」的記錄，確保數據品質，剔除不完整或錯誤的讀數。
  - **數據稀疏性檢查 (`groupby` & `size`)：** 計算每個魚塘每天的記錄數。若某天記錄過少，該數據可能不具代表性。
  - **內連接過濾 (`inner merge`)：** 僅保留滿足「每日最低記錄數」門檻的日期與魚塘，防止後續特徵工程產生偏差。

- `impute_missing_sensor_values(df)`：
  - **目的：** 使用「前向填補法」（Forward-fill）填補感測器欄位的缺失值。
  - **使用的函式與原因：**
    - `sort_values()`: 依據魚塘 ID 與時間排序。對於時序數據，前向填補（使用前一個值）僅在時間序正確時才有意義。
    - `ffill()`: 在各個魚塘組別內，將最後一個有效觀測值向下傳遞到下一個缺失值。
    - **為何使用：** IoT 數據通常是連續的。若感測器暫時斷線，環境條件在短時間內通常不會劇烈變化，前向填補提供了一個合理的估計值。
    - `bfill()`: （選擇性）用於填補序列開頭的缺失值，確保數據盡可能完整，以便進行數值計算。

- `handle_outliers(df)`：
  - **目的：** 識別並修正物理上不合理（Physically Implausible）的讀數，特別是針對 pH 值與深層水溫。
  - **使用的函式與原因：**
    - **數值替換 (`df.loc`)：** 將超出物理常規範圍（如 pH 0-14 之外）的數值替換為 `None`（即 NaN）。
    - **為何使用：** 感測器故障可能產生極端且不可能的讀數，將其轉為 NaN 後，可使用更穩健的方法重新估計，而非直接刪除整筆記錄。
    - **線性插值 (`interpolate(method='linear')`)：** 使用線性插值填補因異常值處理產生的空缺。
    - **為何使用：** 線性插值假設有效點之間是逐漸變化的，這非常符合環境參數（如溫度、pH）的實際物理特性。